

AI Support Engineer Copilot

Using LangGraph and LangSmith to Automate Support Escalation Summary

ASHISH SHAH



What I Built



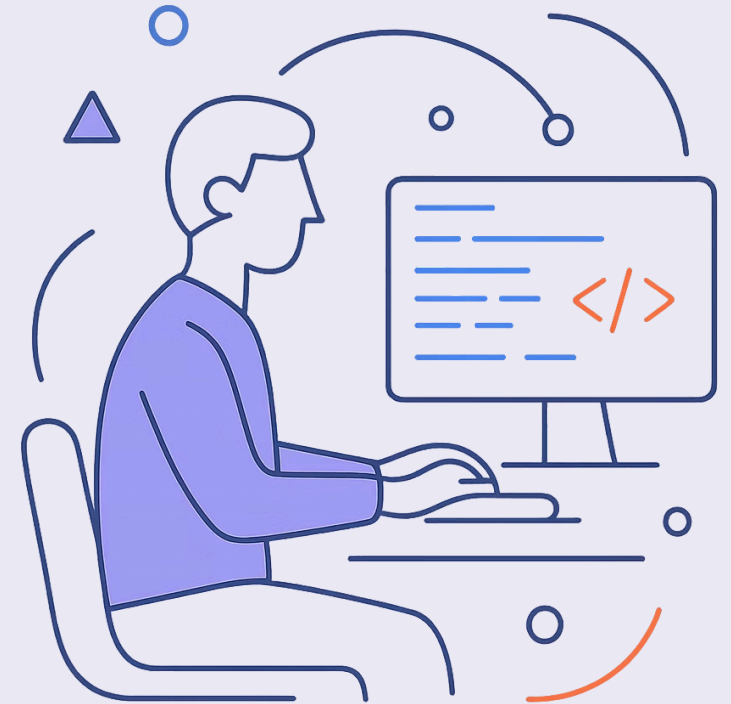
LangGraph



LangSmith



Learnings



Assignment Requirements vs. What I Delivered

Requirements

- Build a workflow using LangGraph
- Use nodes, edges, and state
- Use LangSmith tracing
- Create an evaluation dataset
- Run experiments with evaluators
- Share learnings and friction points

Delivered

- Multi-node LangGraph workflow with conditional routing
- LangSmith tracing
- Dataset with 25 support issues across 5 categories
- Deterministic + LLM-as-Judge evaluation
- Summary of learnings and challenges

The Problem

Support engineers perform the same initial steps for every ticket before escalating.

Typical Manual Process



Challenges

Repetitive Work

Same steps run for every ticket

Inconsistency

Troubleshooting quality varies by engineer

Escalation Quality

Summaries differ in depth and clarity

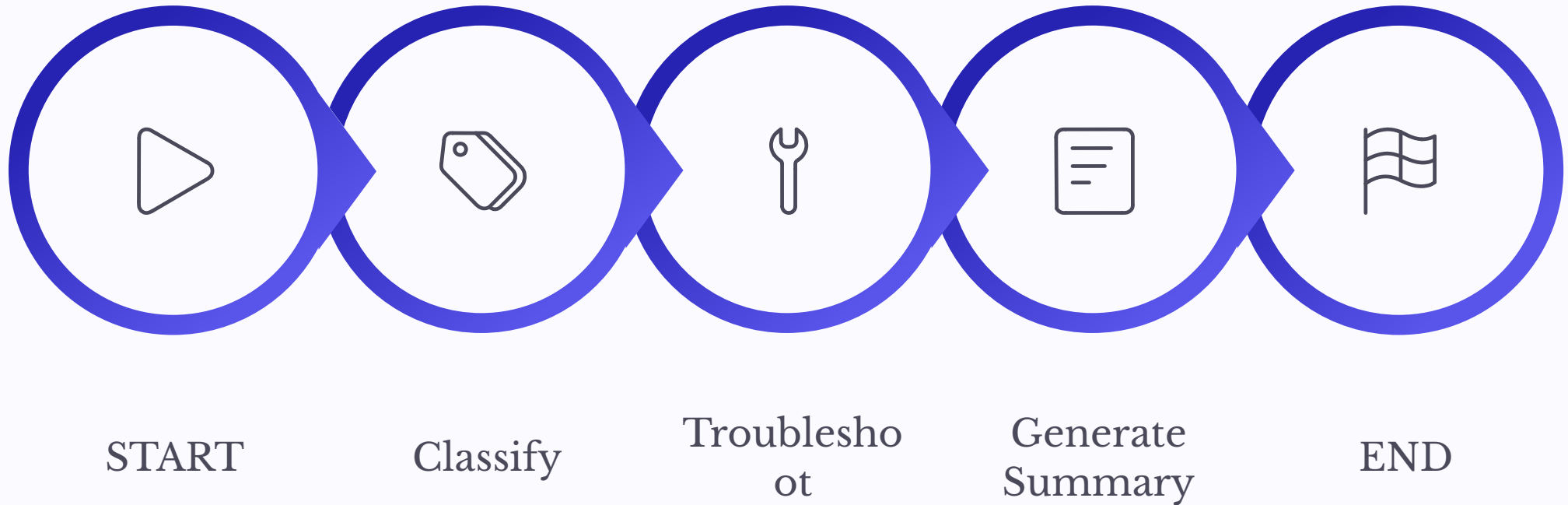
Scale

Difficult to handle high ticket volume

 Goal: Help support engineers investigate issues, recommend troubleshooting actions, and prepare consistent escalation summaries.

Version 1: Simple Linear Workflow

The first version was intentionally simple.

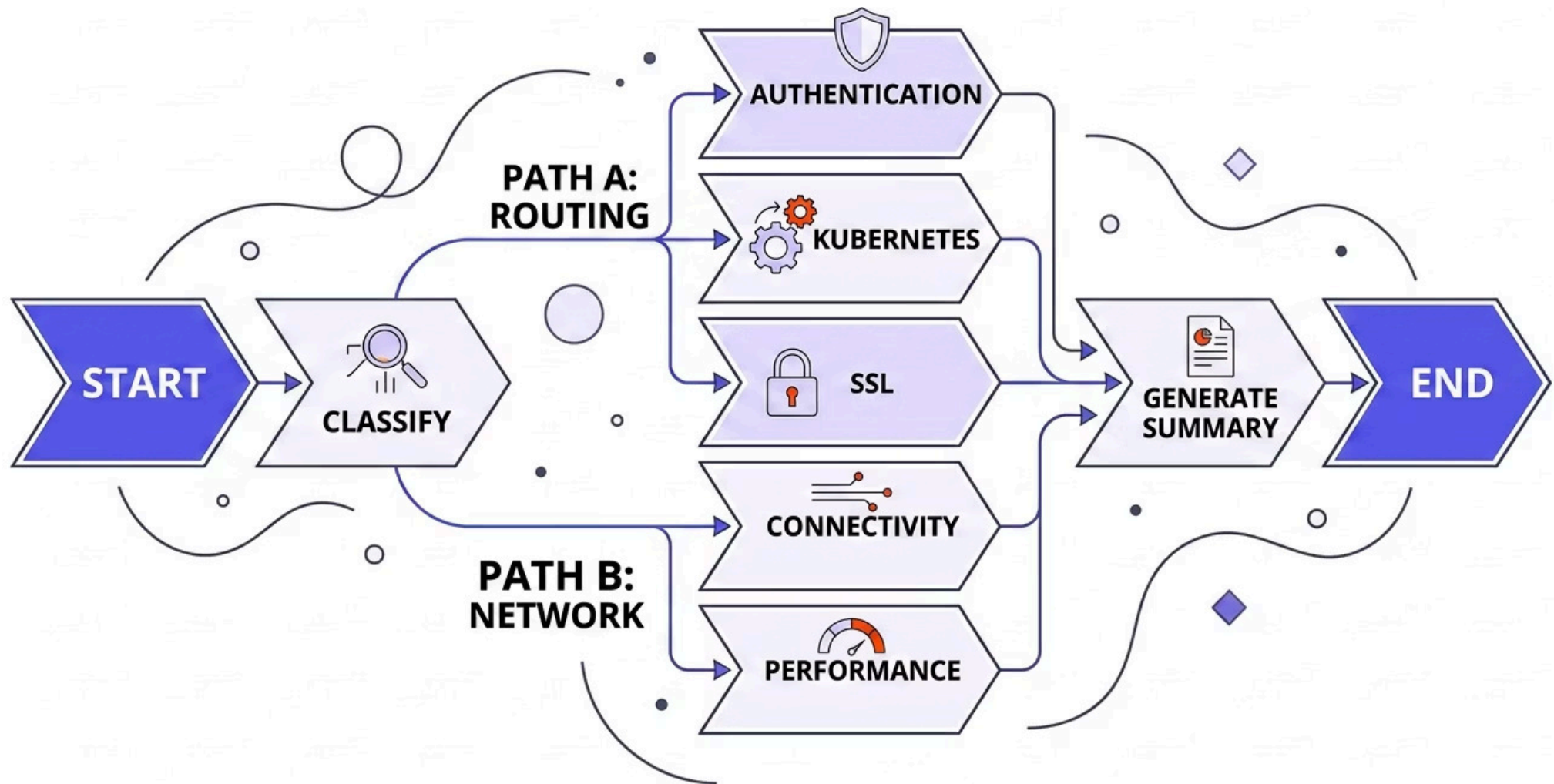


✓ It worked! Every issue went through classification, troubleshooting, and summary generation

⚠ Challenge: Every issue followed the same troubleshooting path

How the Design Evolved

Instead of sending every issue through the same troubleshooting process, use conditional routing.



- ❑ This felt much closer to a real support workflow where each category gets its own dedicated troubleshooting path.

Solution Architecture

Incoming Issue

Raw support ticket text enters the workflow

Classification Node

Use GPT-4o-mini to categorize the issue type

Conditional Routing

Route the issue to the appropriate troubleshooting node

Troubleshooting Node

Generates targeted remediation steps

Summary Generation

Produces a structured escalation summary

The workflow was easy to follow and easy to visualize. Information moved clearly through each step with full state visibility at every node.

LangGraph Core Concepts



State

Shared information passed between nodes. The backbone of the entire workflow.



Node

A single unit of work. Each node reads state, performs an action, and writes back.



Edge

A connection between nodes defining the flow of execution.



Conditional Edge

Allows dynamic routing based on workflow decisions. The main part of branching logic.



START and END

Define workflow boundaries and entry/exit points.



Biggest Learning: Understanding how state moves through the workflow took some time to fully grasp

Project Structure

Repository Layout

```
ai-support-copilot/  
├── README.md  
├── requirements.txt  
├── src/  
│   ├── state.py  
│   ├── nodes.py  
│   ├── graph.py  
│   ├── main.py  
│   └── evaluators.py  
├── data/  
└── docs/
```

Key Files

state.py

Defines shared workflow state schema

nodes.py

The business logic: classify, route, troubleshoot, summarize

graph.py

The LangGraph workflow

evaluators.py

Deterministic and LLM-as-Judge evaluators

✓ Each file has a specific responsibility. This made the project organized and easy to understand

Classification Node

`classify_issue()`

This node receives the support issue, determines which category it belongs to, and updates workflow state.

Model: GPT-4o-mini

Classification felt like a natural use case for an LLM and allowed me to focus on workflow design rather than building custom classification logic.

Possible Output Categories



Authentication



SSL



Kubernetes



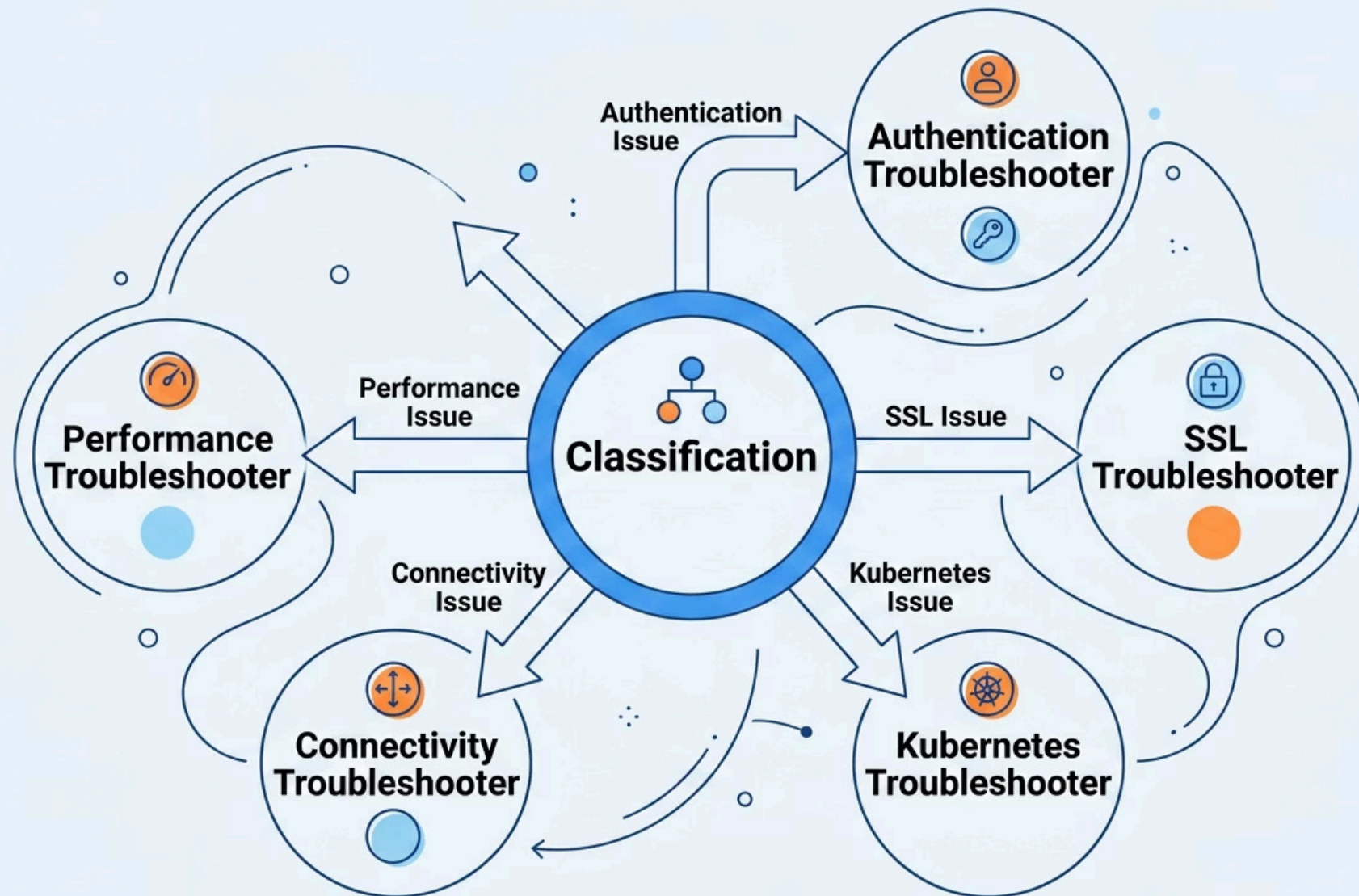
Connectivity



Performance

Routing Logic

The `route_issue()` function uses `add_conditional_edges()` to direct each classified issue to its specialized troubleshooting node.



 Different issue types require different troubleshooting approaches. Routing allowed each category to have its own specialized path.

Troubleshooting Nodes

Five Specialized Nodes

01

`auth_troubleshooter()`

Verify API key validity, check Authorization header, review auth logs

02

`ssl_troubleshooter()`

Check certificate expiration, verify chain, validate trust store

03

`kubernetes_troubleshooter()`

Inspect pod status, check resource limits, review events

04

`connectivity_troubleshooter()`

Test DNS resolution, verify firewall rules, check routing

05

`performance_troubleshooter()`

Analyze CPU/memory, review query plans, check resource saturation

Design Decision

- ❑ Troubleshooting steps were intentionally kept simple and hardcoded. The focus was on workflow design.

Future: replace with retrieval from a knowledge base



Summary Generation Node

`generate_summary()`

The final node produces a structured escalation summary before handing off to a human engineer. The summary includes:

1	2
Problem Plain-language description of the issue	Category Classified issue type
3	4
Actions Taken Troubleshooting steps executed	Recommended Next Step Suggested escalation path

Why This Matters

Support engineers need to summarize findings before escalating. This node creates a consistent, structured escalation summary every time regardless of which engineer is on shift.

Example Workflow Execution

Input

Users started receiving 401 errors after rotating API keys.

Execution Path

START
Begin workflow
execution

Troubleshoo
ter
Run
authentication
checks

END
Complete
workflow

Classify
Category:
Authentication

Generate
Summary
Compile findings
and actions

Output Summary

Category

Authentication

Actions Taken

- Verify API key validity
- Check Authorization header format
- Review authentication logs for errors

Recommended Next Step

Investigate API key rotation configuration and propagation delay.

How I Used LangSmith

LangSmith became the primary debugging and observability tool during development.



Trace Execution

Follow workflow execution step-by-step and verify routing decisions in real time



Inspect I/O

Review exact inputs and outputs at every node to catch unexpected behavior



Latency Review

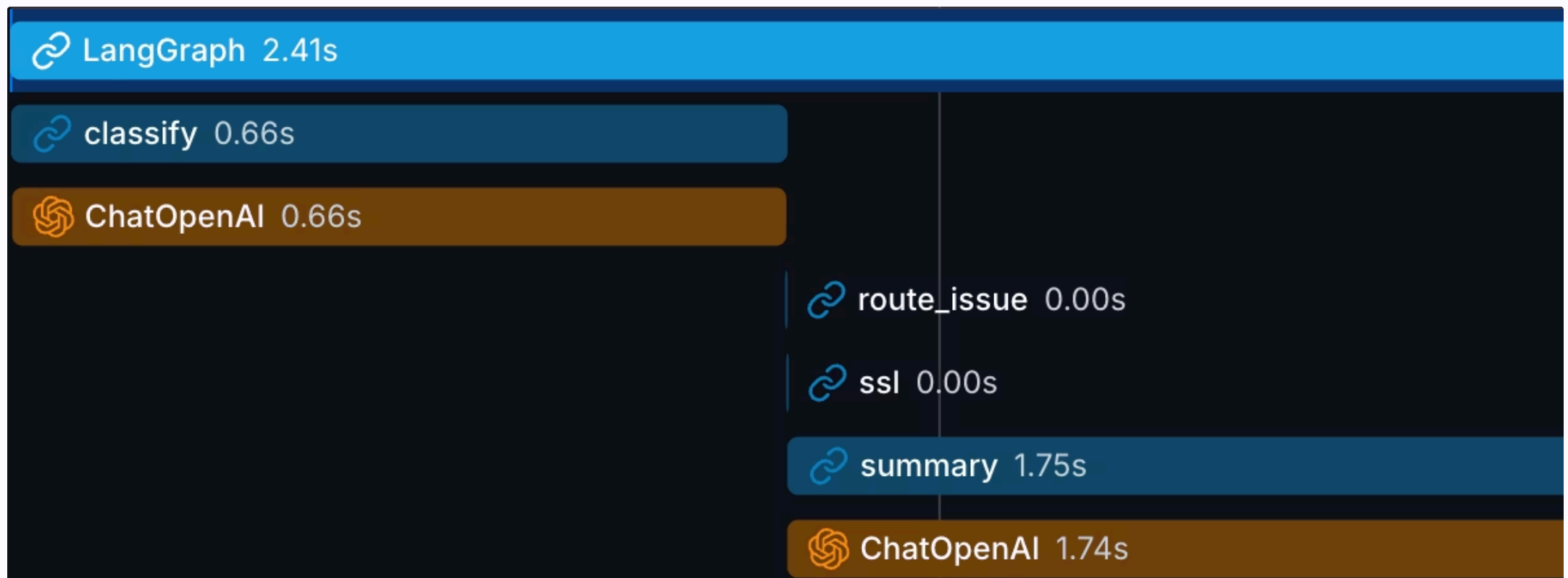
Measure per-node latency to identify bottlenecks in the pipeline



Debug Behavior

Pinpoint exactly where and why unexpected workflow behavior occurred

Example LangSmith trace showing workflow execution from classification through summary generation.



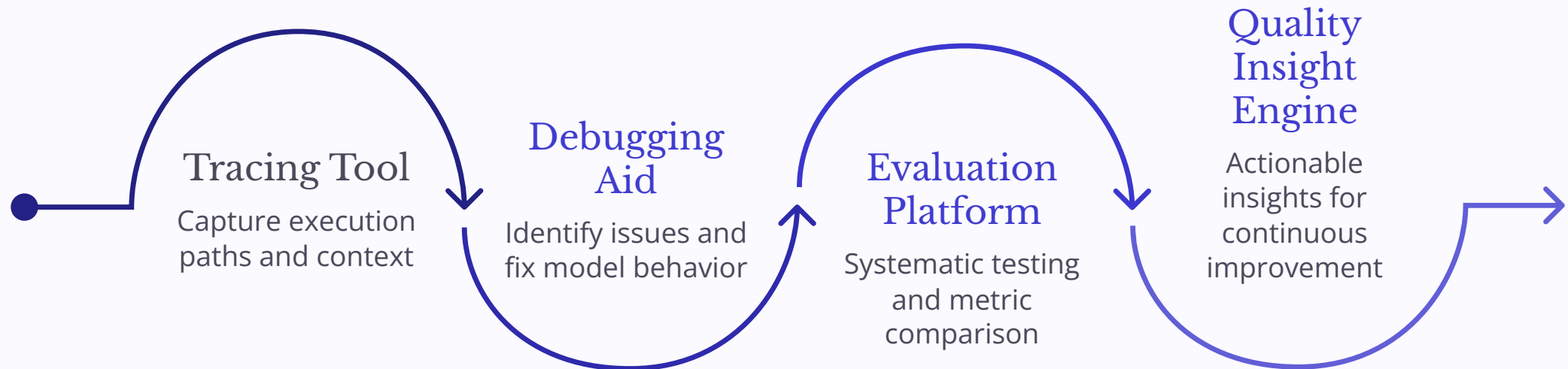
What Surprised Me About LangSmith

Initial Expectation

LangSmith would mostly help with tracing as a useful debugging tool. Before creating the dataset, I was mostly testing individual examples and checking whether the results looked reasonable.

What Actually Happened

Once I started running experiments against a larger set of issues, I could actually measure performance and identify areas where the workflow could be improved. Evaluation gave me much better insight into workflow quality.



✓ Evaluation became the most valuable part of the LangSmith integration along with tracing.

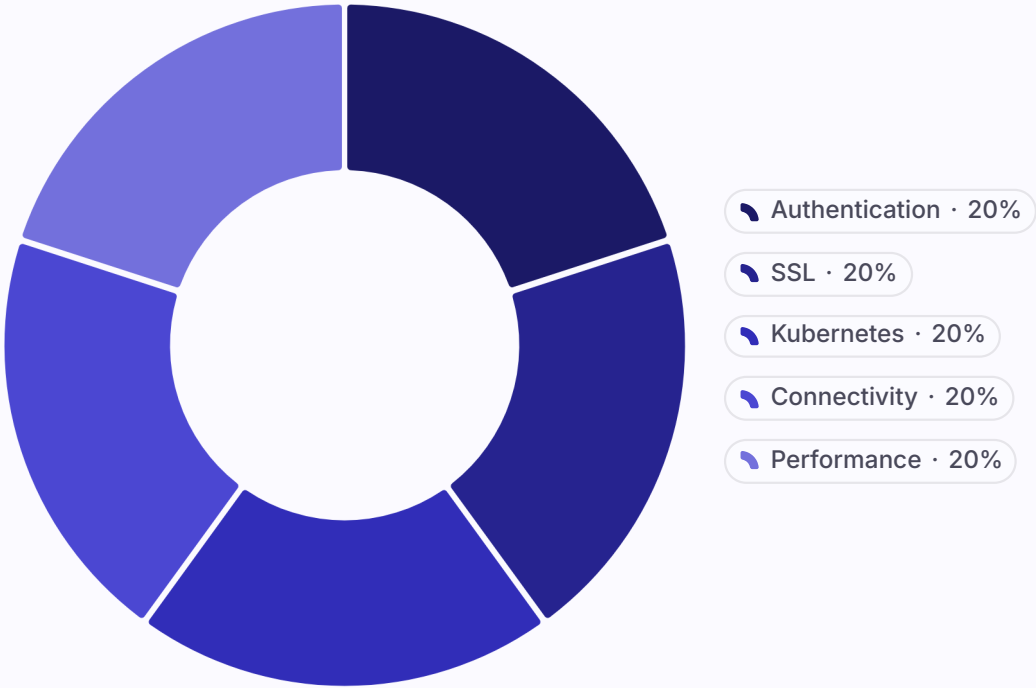
Evaluation Dataset

To evaluate the workflow, I created a dataset with 25 support issues across five categories.

25
Total Issues
In the evaluation dataset

5
Categories
Evenly distributed, 5 each

Category Breakdown



 The dataset gave me a consistent and repeatable way to measure performance and compare results as the workflow evolved.

Evaluation Strategy

I used two evaluation approaches.

Deterministic Evaluation

Metric: Category Accuracy

```
predicted == expected
```

Binary pass/fail check. Fast, objective, and repeatable. Ideal for classification correctness where ground truth is known.

LLM-as-Judge Evaluation

Metric: Summary Quality

An LLM evaluates whether the generated escalation summary is complete, accurate, and actionable.

Captures the quality essential for open-ended generated text.

- ❏ Different parts of an AI workflow often require different evaluation methods. Using both approaches together gives a complete picture: correctness and quality.

Experiment Results

LANGSMITH EXPERIMENT: SUPPORT-ESCALATION-V2

96%

Category Accuracy

24 of 25 issues correctly classified

100%

Summary Quality

All summaries passed LLM-as-Judge evaluation

0%

Error Rate

Zero workflow execution failures

LangSmith experiment results showing category accuracy, summary quality, and overall workflow performance.



Interesting Finding: Ambiguous Classification

The Edge Case

Node memory pressure is affecting workloads.

Expected

Kubernetes

Predicted

Performance

The issue could reasonably fit both categories. "Node memory pressure" is a Kubernetes concept, but "memory pressure affecting workloads" reads as a performance symptom.

What I Learned From This Result

- Most issues were classified correctly.
- One issue produced an unexpected result.
- That result helped identify an area where the workflow could be improved.
- This reinforced the value of running evaluations against a larger dataset.

✓ The evaluation measured quality which can drive improvement.



Learnings & Friction Points

What Took Time

State Management

Understanding how state flows and changes between nodes

Conditional Routing

Wiring `add_conditional_edges()` correctly the first time

Datasets vs Local Testing

A total shift in mindset and approach

Evaluation Approaches

Knowing when to use deterministic vs LLM-as-Judge

What Helped

Build Incrementally

Start simple, add complexity only when the foundation works

Review Traces

LangSmith traces helped understand what was happening at each step vs print stmts.

Run Experiments

Systematic evaluation revealed issues invisible to manual testing



Most Valuable Lesson:

Evaluation became much more important than expected. It becomes increasingly important as workflows become more sophisticated.



What I Would Build Next

A few key areas for future development and expansion of the AI Support Engineer Copilot.

1

Short Term

- Improve classification prompts
- Add more issue categories
- Expand dataset size

2

Medium Term

- Dynamic troubleshooting generation
- More realistic troubleshooting workflows

3

Long Term

- RAG Retrieval
- Vector Database

Final Takeaway

This project helped me understand how LangGraph and LangSmith work together to build robust and observable AI applications.



LangGraph

Provided the workflow orchestration



LangSmith

Helped me trace, debug, and evaluate the workflow behavior and quality.



LangGraph + LangSmith

The combination made it much easier to understand, iterate on, and improve how the workflow behaved and performed.

